

Codebase Documentation and Implementation Details

Martin Kaae Jensen

June 12, 2026

Contents

1	Introduction	3
2	Requirements	3
3	Main Algorithms Overview	3
3.1	State Space Recursion (SSR)	3
3.2	Model-Adapted CMA-ES	4
4	Loss Function Implementation	5
4.1	Core Algorithm	5
4.2	Core Components	5
4.2.1	Continuation Utility Computation	5
4.2.2	Loss Computation	6
4.2.3	Best Response Extraction	6
4.3	Implementation Variants	6
4.3.1	Standard Implementation (<code>Lossfunction.m</code>)	6
4.3.2	SSR Implementation (<code>algorithm_ssr.m</code>)	7
4.4	Features	7
4.4.1	Performance Optimizations	7
4.4.2	Numerical Safeguards	7
5	SSR Algorithm Details	7
5.1	Overview	7
5.2	Core Algorithm	8
5.3	Implementation Details	8
5.3.1	Algorithm Specific Parameters	8
5.3.2	Search Space Control	9
5.3.3	Solution Options	9
5.4	Model-Specific Defaults	9
5.5	Model-Specific Applications	10
5.6	Theoretical Foundations	10
6	CMA-ES Algorithm Details	11
6.1	Basic Mechanics	11
6.2	Transformation Handling	11

6.2.1	Overview	11
6.2.2	Implementation Details	11
6.2.3	Default vs TSA Variants	12
6.3	Implementation Variants	12
6.3.1	Standard Implementation	12
6.3.2	Hansen’s Implementation	13
6.3.3	Separable Implementation	13
6.3.4	Active Implementation	14
6.3.5	TSA Implementation	14
6.4	Additional Features	14
6.4.1	Policy Injection	14
6.4.2	Gradient Injection	14
6.5	Implementation Details	14
6.5.1	Common Elements	14
6.5.2	Other Differences	15
7	Model-Specific Implementations	15
7.1	Deterministic Consumption-Saving Models (Section 4)	15
7.2	Deterministic Optimal Growth Models (Section 3)	15
7.3	Bewley-Aiyagari IID Models (Sections 1 and 5)	16
7.3.1	Near RBC Example (Section 1)	16
7.3.2	IID Models with Mandatory Saving (Section 5)	16
7.4	Two-State Bewley-Aiyagari Models (Section 5)	16
7.5	Additional Models (Not in Paper)	17
7.5.1	Brock-Mirman Model	17
8	Code Organization by Model Type	17
8.1	Deterministic Optimal Growth/Social Planner Models	17
8.2	Stochastic Optimal Growth Models (“Quasi-Hyperbolic Brock-Mirman”)	17
8.3	Deterministic Consumption-Saving Models	18
8.4	Bewley-Aiyagari Models	18
9	Code Placement by Figure	18
9.1	Section 1: Near-RBC Example	18
9.2	Section 3: Deterministic Optimal Growth	19
9.3	Section 4: Deterministic Consumption-Saving Models	19
9.4	Section 5: Bewley-Aiyagari Models	19

1 Introduction

This document describes the shared components and model-specific adaptations across the codebase implementing State Space Recursion (SSR) and Model-Adapted CMA-ES algorithms for computing Markov perfect saving functions in dynamic economic models. Please note that new models are on occasion added to the repository (this documentation focuses on the original codebase).

2 Requirements

Requirement	Notes
MATLAB R2024b or newer	Tested with R2024b & R2026a.
Statistics & Machine Learning Toolbox	Needed for <code>mvnrnd</code> in CMA-ES routines.
Optimization Toolbox (optional)	Required for <code>fminbnd</code> when <code>directsearch=0</code> . Not used by default.
Parallel Computing Toolbox (optional)	<code>parfor</code> used in <code>algorithm_call_*.m</code> . Change to <code>for</code> if absent.
System Requirements	
Minimum CPU	Mobile Intel Core i5 or AMD equivalent
Minimum RAM	4GB
Storage	2GB+ free disk space
Note	Bewley-Aiyagari models are computationally intensive and <i>e.g.</i> 16GB ram + true parallelization such as AMD Ryzen 7 5800X or Intel Core i7-12700K will reduce run time by approximately 95 % compared to a Mobile Intel Core i7 Processor or equivalent

3 Main Algorithms Overview

3.1 State Space Recursion (SSR)

Primary use: Deterministic models (Sections 3 and 4 in the paper)

SSR iteratively minimizes the Loss function (detailed in Section 4 below) through direct policy updates:

- Uses direct utility summation in deterministic cases for higher precision
- Falls back to value function iteration for stochastic cases
- Employs specialized grid construction optimized for deterministic settings
- Evaluates candidate policies using Lossfunction as detailed below

Full implementation details are provided below in Section 5. The theoretical underpinnings are found in the paper's Section 3.3.1.

3.2 Model-Adapted CMA-ES

The Model-Adapted Covariance Matrix Adaptation Evolution Strategy (CMA-ES) minimizes the Loss function (detailed in Section 4) through population-based search:

1. Samples candidate solutions from a multivariate normal distribution
2. Updates the distribution parameters based on successful candidates (measured by Loss function values)
3. Adapts both the covariance matrix and step size
4. Converges when the Loss function approaches zero, indicating a Markov perfect policy

The method builds on the CMA-ES algorithm introduced in Hansen, N. and A. Ostermeier (2001): “Completely Derandomized Self-Adaptation in Evolution Strategies”, *Evolutionary Computation* 9, 159–195, with adaptations specific to economic models. Full implementation details, including transformation handling and variants, are provided in Section 6.

Model-specific implementations and adaptations of both algorithms are detailed in Section 7.

4 Loss Function Implementation

4.1 Core Algorithm

The Loss function (objective function of the Approximate Loss Representation in Section 3.2 of the paper) is the objective function minimized by both SSR and CMA-ES algorithms. It is implemented in `Lossfunction.m` across all model types, with a specialized version for deterministic cases in `algorithm_ssr.m`.

While the actual implementation is heavily optimized with vectorization, caching, and numerical safeguards that make the code complex, the core algorithm is straightforward. Given a first-mover's policy g represented by its values x on the asset grid:

```
For each state a:
  1. Compute continuation value function  $V_g$ :
     - In deterministic case: Sum of discounted utilities
     - In stochastic case: Value function iteration
     (Implemented in valf/valf2 functions)

  2. Determine second-mover's optimal policy:
     - Grid search over feasible saving levels  $s$ 
     - For each  $s$ , compute  $u(f(a) - s) + V_g(s)$ . Note that in stochastic
       models, the state generally includes the productivity (meaning
       that a separate policy function is determined for each
       productivity level)
     - Find maximum payoff value
     - Among all  $s$  achieving this maximum payoff, select the highest one
       as  $br(a, l)$  where  $l$  is the productivity
     (Computed within Lossfunction.m)

  3. Calculate loss at asset state (grid point)  $a$  and productivity  $l$ :
     - Compute  $L(a, l) = [u(f(a, l) - br(a, l)) + V_g(br(a, l))] - [u(f(a, l) - x(a, l)) + V_g(x(a, l))]$ 
     - If  $L(a, l) < 0$ , set  $L(a, l) = 0$  and  $br(a, l) = x(a, l)$ 
     (Computed within Lossfunction.m)

  4. Sum losses across asset grid and productivity states to obtain (
     total) loss:  $loss = \sum(L(a, l))$ 

Return:  $[loss, br]$  where  $br$  is the best response (second-mover's policy)
```

4.2 Core Components

4.2.1 Continuation Utility Computation

- Implemented in `valf2` (two-state Bewley-Aiyagari model) and `valf` (all other models)
- Computes continuation utility V_g as defined in Section 2.2 in the paper
- Uses persistent variables to cache results between calls (enables warm starts by reusing previous value function)
- Uses linear interpolation (piecewise linear approximation on $[a, b]$)
- Policy extrapolation beyond upper bound:

- Two-state model: Sophisticated extrapolation using extended grid and supervised policy iteration (see `extended_policy` in `aiyagari_QH_main_calibration.m`)
- All other models: Linear extrapolation using average slope of last 10 policy segments

Key parameters:

- `iterlimit`: Maximum number of value function iterations (default 100). Consider raising to 200 or more if high precision is important (an example where high precision is not important is when distributions are studied in Bewley-Aiyagari models as these stabilize much before means as precision increases; an example where high precision may be important is when questions relating explicitly to the mean are studied)
- `tol`: Convergence tolerance, imported from main script. Lower values require more iterations. Similar considerations to `iterlimit` apply (reducing tolerance will typically require raising `iterlimit` or iteration will terminate without tolerance being satisfied)
- `maxPoints`: Maximum number of grid points. Default $2n$ where n is number of grid intervals

Note: When `tol = 1e-20`, `iterlimit` is automatically increased to ensure convergence.

4.2.2 Loss Computation

- Solves second-mover's optimization problem via direct search (default) or `fminbnd`
- Computes state-specific losses
- Sums losses across states to obtain total loss
- Includes safeguards for numerical stability and boundary cases

Key parameters:

- `dsp`: Direct search precision (number of points per grid interval)
- `dissaving`: Controls grid construction for optimization. When 1, restricts search to dissaving policies (up to current assets). When 0, allows saving up to current income.
- `directsearch`: When 1 (default), uses direct grid search. When 0, switches to `fminbnd` optimization. The `fminbnd` option is mainly useful for debugging as direct search is both faster and more precise.

4.2.3 Best Response Extraction

- Returns the maximizing policy from the second-mover's optimization
- When multiple policies achieve the same value (ties), selects the highest feasible saving value
- Implemented through columnwise maximum over masked objective values
- Uses numerical tolerance (10^{-14}) for robust tie detection

4.3 Implementation Variants

4.3.1 Standard Implementation (`Lossfunction.m`)

- Base implementation for all model types

- Handles both deterministic and stochastic cases
- Uses grid search by default for robustness
- Includes both direct search and `fminbnd` optimization methods

4.3.2 SSR Implementation (`algorithm_ssr.m`)

- For deterministic cases ($p = 0$ or $p = 1$): Uses direct summation of discounted utilities for higher precision than value function iteration
- Falls back to value function iteration for stochastic cases
- Specialized grid construction optimized for deterministic case

4.4 Features

4.4.1 Performance Optimizations

- Direct grid search for global optimization
- Vectorized operations where possible
- Adaptive grid refinement
- Early termination for convergence
- Caching of value functions between calls

4.4.2 Numerical Safeguards

- Handling of boundary cases (especially at $a = 0$)
- Prevention of negative consumption
- Management of non-real or negative losses
- Grid refinement based on value function gradients
- Robust tie-breaking in optimization

5 SSR Algorithm Details

5.1 Overview

The State Space Recursion (SSR) algorithm, introduced in Section 3.3.1 of the paper, is a specialized method for computing Markov perfect saving functions focused primarily on deterministic models.¹ Unlike the Model-Adapted CMA-ES, which is highly adaptable and can also be used for a variety of models not discussed in the paper/codebase, SSR is a tailored algorithm that:

1. Avoids calling on global optimization methods
2. Is computationally extremely cheap/efficient with a minimal memory footprint (because of this, on a fast CPU there is essentially no limit on the precision that can be attained, that is, on the number of grid points)

¹The algorithm also functions in stochastic models but the theoretical foundation is not clear.

3. Guarantees convergence to an approximate Markov policy
4. Starts from a steady state (typically the borrowing limit in consumption-saving models). Note that in cases where the borrowing limit is not the relevant starting point, steady states can be computed very cheaply and robustly using the steady state computation code included in the code base.

5.2 Core Algorithm

The algorithm’s key insight, as detailed in Section 3.3.1, is that we can construct Markov perfect policies recursively by:

1. Starting from a steady state (x^*)
2. Building the policy function one grid point at a time
3. Using piece-wise linear approximations between grid points
4. At each new grid point x_j :
 - Takes as given the policy values computed at previous grid points
 - Finds the unique value that makes the second-mover’s best response match the first-mover’s policy
 - Uses this value to extend the piece-wise linear approximation
 - Continues until reaching the upper bound or crossing the 45-degree line

5.3 Implementation Details

The algorithm is implemented in `algorithm_ssr.m` and is configured through several key parameters that are set in the main model scripts just before calling the algorithm:

5.3.1 Algorithm Specific Parameters

- **branch** (set in main script): Controls computation method
 - If 0: Uses value function iteration
 - If 1: Uses direct utility summation for deterministic case ($p = 0$ or $p = 1$), value function iteration otherwise. Cannot be used in stochastic models.
- **con** (set in main script): Controls continuation policy
 - If 0: Uses constant policy for future periods
 - If 1: Uses provided policy (opts) for future periods
- **cct** (set in main script): Controls console output
 - If 0: No progress display
 - If 1: Shows iteration progress

5.3.2 Search Space Control

- **dissaving**: Controls search space and precision
 - If 1: Optimizes search by limiting to weak dissaving policies
 - If 2: Same as 1, plus additional precision check against 45-degree line (helpful if policies touch the 45-degree line/there are steady states). Only used in `determ_cons_savmgr.m`

5.3.3 Solution Options

- **ssrdc**: Controls computation of dynamically consistent solution
 - If 1: SSR computes both DC and naive solutions
 - If 0: Uses Kaplan’s EGP algorithm for DC solution

5.4 Model-Specific Defaults

The SSR algorithm is used in the deterministic consumption-saving models from Section 4 of the paper:

1. `determ_cons_sav0_93.m`
 - `dissaving = 1`
 - `branch = 1`
 - `con = 0`
 - Grid: $n = 600$ grid points ($a = 0$, $b = 15$, `gridprec = 40`)
2. `determ_cons_sav0_85.m`
 - `dissaving = 1`
 - `branch = 1`
 - `con = 0`
 - Grid: $n = 600$ grid points ($a = 0$, $b = 15$, `gridprec = 40`)
3. `determ_cons_sav1_1.m`
 - `dissaving = 1`
 - `branch = 1`
 - `con = 0`
 - Grid: $n = 600$ grid points ($a = 0$, $b = 15$, `gridprec = 40`)
4. `determ_cons_savmgr.m`
 - `dissaving = 2`
 - `branch = 1`
 - `con = 0`
 - Grid: $n = 1000$ grid points ($a = 0$, $b = 10$, `gridprec = 100`)

5. `determ_cons_sav0_85largerdelta.m`

- `dissaving = 1`
- `branch = 1`
- `con = 0`
- Grid: $n = 600$ grid points ($a = 0$, $b = 15$, `gridprec = 40`)

5.5 Model-Specific Applications

The SSR algorithm is used exclusively in the deterministic consumption-saving models from Section 4 of the paper:

1. `determ_cons_sav0_93.m`: Moderate Present Bias
 - Demonstrates periodic catchup with dynamically consistent solution
 - Shows failure of tracking near borrowing limit
 - Used for Figures 7A, 8A
2. `determ_cons_sav0_85.m`: Substantial Present Bias
 - Illustrates expanded region of naive-like behavior
 - Shows weakening of sophisticated households' self-enforcement
 - Used for Figure 7B
3. `determ_cons_sav1_1.m`: Future Bias
 - Shows qualitative structure remains unchanged
 - Shifts from overconsumption to underconsumption
 - Used for Figures 7C, 8C
4. `determ_cons_savmgr.m`: Modified Golden Rule
 - Interest rate equals inverse of discount factor
 - Shows periodic dissaving with upward jumps realigning to first-best
 - Used for Figures 7D, 8D
5. `determ_cons_sav0_85largerdelta.m`: Higher Long-run Discount
 - Examines case with substantial long-run patience
 - Shows welfare implications at different asset levels
 - Used for Figure 8B

5.6 Theoretical Foundations

The algorithm's correctness is established in the paper through:

1. Theorem 4: Proves the algorithm produces a valid solution to the approximated problem

2. Lemma “zero loss lemma”: Guarantees existence of solution with zero loss
3. Various supporting results about monotonicity and piece-wise linear approximations

For full mathematical details and proofs, please refer to the paper.

6 CMA-ES Algorithm Details

6.1 Basic Mechanics

CMA-ES (Covariance Matrix Adaptation Evolution Strategy) iteratively:

1. Samples candidate solutions from a multivariate normal distribution
2. Updates the distribution parameters based on successful candidates
3. Adapts both the covariance matrix and step size

6.2 Transformation Handling

6.2.1 Overview

CMA-ES performs best in unconstrained parameter spaces. However, in our setting:

1. Markov policies are restricted to bounded intervals $[a, b]$
2. From Section 3, we know Markov policies are monotone, which allows us to restrict the search space and improve efficiency

The transformation system handles both aspects by converting between:

- Policy Space (x): Bounded Markov policies
- Parameter Space (z): Unconstrained space where CMA-ES operates

This approach is used consistently across all model variants (IID, two-state, deterministic) with appropriate adjustments for each model’s specific requirements.

6.2.2 Implementation Details

ztox (Parameter Space $\rightarrow$ Policy Space)

```
function x = ztox(z, a, b)
    x1 = a + (b - a)/(1 + exp(-z(1)))      % Logistic (first coord.)
    alphas = exp(z(2:end))                  % Exponential of increments
    // ... compute cumulative sums for monotonicity ...
```

- Takes unconstrained parameters z and maps them to a Markov policy x
- Uses logistic function to map first point to $[a, b]$ interval
- Uses exponential transform to ensure positive increments (exploiting monotonicity)
- For two-state models, handles both low and high income states separately
- Chosen because logistic and exponential transforms naturally map $\mathbb{R}$ to bounded intervals and positive reals respectively, while being smooth and invertible

xtoz (Policy Space $\rightarrow$ Parameter Space)

```
function z = xtoz(x, a, b)
    frac = (x(1) - a) / (b - a)           % Inverse logistic (first coord.)
    z(1) = log(frac / (1 - frac))
    D = diff(x)                           % Compute increments
    z(2:end) = log(D)                     % Log transform of increments
```

- Inverts the transformation to map policies back to unconstrained space
- Uses logit (inverse logistic) for first point
- Takes logs of policy increments
- Includes numerical safeguards (epsilon clamping) to avoid singularities
- Natural inverses of the forward transforms, chosen to maintain differentiability which is important for CMA-ES's covariance adaptation

6.2.3 Default vs TSA Variants

Some models offer a TSA (Two-Step Ahead) variant that provides an alternative way to structure the search space. The key insight is that:

1. Best responses in policy space are naturally monotone
2. Two-step ahead responses preserve monotonicity (monotone response to monotone input)

This creates a particularly well-behaved optimization landscape without requiring explicit monotonicity constraints in the transformation. The transformation mechanics remain the same, but the underlying space being searched becomes more structured.

Different economic models in the codebase are configured to use specific variants by default, based on extensive testing. When implementing a new model or facing convergence challenges, experimenting with both variants and using restarts can be beneficial. Restarts (setting `loadoutput=1` in the main script after some improvement) reset the CMA-ES step length, which can help escape local minima.

Examples of default configurations:

- The near-RBC model (Bewley-Aiyagari iid --- Sec 1 and Sec 5/Near RBC Textbook Example --- S) uses the TSA variant
- The two-state model (Bewley-Aiyagari general two-state --- Sec 5/mainscripts/aiyagari.m) uses the default variant

6.3 Implementation Variants

6.3.1 Standard Implementation

The standard implementation follows Hansen, N. and A. Ostermeier (2001): “Completely Derandomized Self-Adaptation in Evolution Strategies”, *Evolutionary Computation* 9, 159–195.

Used in:

- Deterministic optimal growth models (Section 3)

- Brock-Mirman models (not in paper)
- Deterministic consumption-saving models (Section 4)

Key features:

- Direct covariance matrix updates
- Uses eigendecomposition for numerical stability
- Stores evolution paths in cell arrays
- Simplest implementation

Reference: Hansen & Kern (2004, “Evaluating the CMA Evolution Strategy on Multimodal Test Functions”, PPSN VIII).

6.3.2 Hansen’s Implementation

Reference: Hansen, N. (2006): “The CMA Evolution Strategy: A Comparing Review”, Towards a New Evolutionary Computation, 75–102.

Used in:

- Bewley-Aiyagari general two-state models (Section 5)
- Bewley-Aiyagari IID models (Sections 1 and 5)

Key features:

- Uses Cholesky factorization for covariance updates
- Maintains $C = A \cdot A^T$ representation
- Performs periodic Cholesky decomposition for numerical stability
- More efficient for high-dimensional problems
- More numerically stable than standard implementation

6.3.3 Separable Implementation

Reference: Ros, R. and N. Hansen (2008): “A Simple Modification in CMA-ES Achieving Linear Time and Space Complexity”, Parallel Problem Solving from Nature X, 296–305.

Used in:

- Alternative implementation for Bewley-Aiyagari general two-state models (Section 5)

Key features:

- Maintains only diagonal elements of covariance matrix
- No rank-one updates ($c_1 = 0$)
- Smaller population size ($\lambda = 3$, $\mu = 2$)
- Linear time and space complexity in problem dimension

- Considerably faster than standard or Hansen implementations in problems where covariance is low

6.3.4 Active Implementation

Reference: Arnold, D.V. and N. Hansen (2012): “A (1+1)-CMA-ES for Constrained Optimisation”, Proceedings of the Genetic and Evolutionary Computation Conference, 297–304.

Only present in the Bewley-Aiyagari general two-state models. Uses finite differences to estimate descent directions.

6.3.5 TSA Implementation

Reference: Hansen, N. and S. Kern (2004): “Evaluating the CMA Evolution Strategy on Multi-modal Test Functions”, Parallel Problem Solving from Nature VIII, 282–291.

Some models offer a TSA (Two-Step Ahead) variant that provides an alternative way to structure the search space. The key insight is that:

1. Best responses in policy space are naturally monotone
2. Two-step ahead responses preserve monotonicity (monotone response to monotone input)

This creates a particularly well-behaved optimization landscape without requiring explicit monotonicity constraints in the transformation. The transformation mechanics remain the same, but the underlying space being searched becomes more structured.

6.4 Additional Features

6.4.1 Policy Injection

Present in all implementations. Periodically injects the best response policy into the candidate pool to accelerate convergence.

6.4.2 Gradient Injection

Only present in the Bewley-Aiyagari general two-state models. Uses finite differences to estimate descent directions.

Reference: Arnold & Hansen (2012, “A (1+1)-CMA-ES for Constrained Optimisation”, GECCO ’12).

6.5 Implementation Details

6.5.1 Common Elements

All implementations share:

- Population size (λ) and parent number (μ) adaptation
- Evolution path tracking
- Step-size control
- Weighted recombination

6.5.2 Other Differences

Stall Detection

- Optimal growth and consumption-saving models: 20 iterations threshold
- Bewley-Aiyagari economies: 50 iterations threshold

Initial Covariance

- Optimal growth and consumption-saving models: Standard identity matrix
- Bewley-Aiyagari general two-state models with TSA: Biased initialization for directed search
- Alternative Bewley-Aiyagari general two-state models: Diagonal initialization

Population Size

- Optimal growth, consumption-saving, and standard Bewley-Aiyagari models: $\lambda = 8$, $\mu = 4$
- Alternative Bewley-Aiyagari general two-state models: $\lambda = 3$, $\mu = 2$

7 Model-Specific Implementations

7.1 Deterministic Consumption-Saving Models (Section 4)

Located in: `determ cons sav` --- Section 4/

Implementation files:

- Main models: `determ_cons_sav0_85.m`, `determ_cons_sav0_93.m`, `determ_cons_sav1_1.m`, `determ_cons_savmgr.m`
- Algorithm: Uses SSR algorithm via `algorithm_ssr.m`

Key features:

- Direct utility summation for higher precision
- Simplified grid structure due to deterministic setting
- Specialized grid construction optimized for deterministic case
- No value function iteration needed

7.2 Deterministic Optimal Growth Models (Section 3)

Located in: `optimal growth` --- deterministic --- Section 3/

Implementation files:

- Main models: `optimal_growth.m`, `optimal_growth_10it.m`
- Algorithm: Uses standard CMA-ES implementation

Key features:

- Standard value function (`valf`)

- Basic policy extrapolation
- Deterministic transitions
- Identity matrix covariance initialization

7.3 Bewley-Aiyagari IID Models (Sections 1 and 5)

Located in: `Bewley-Aiyagari iid --- Sec 1 and Sec 5/`

7.3.1 Near RBC Example (Section 1)

Located in: `Near RBC Textbook Example --- Sec 1/`

Implementation files:

- Main models: `aiyagari_Sec1_PanelA.m`, `aiyagari_Sec1_PanelB.m`
- Algorithm: Uses Hansen's CMA-ES implementation with TSA variant

Key features:

- Standard value function (`valf`)
- Basic policy extrapolation
- IID state transitions
- Optimized for near-RBC parameterization

7.3.2 IID Models with Mandatory Saving (Section 5)

Located in: `Aiyagari iid and mandatory sav schemes --- Sec 5/`

Implementation files:

- Main models: `aiyagari_iid.m`, `aiyagari_iidsm.m`
- Algorithm: Uses Hansen's CMA-ES implementation

Key features:

- Standard value function (`valf`)
- Basic policy extrapolation
- IID state transitions
- Specialized handling for mandatory saving constraints

7.4 Two-State Bewley-Aiyagari Models (Section 5)

Located in: `Bewley-Aiyagari general two-state --- Sec 5/`

Implementation files:

- Main calibration: `aiyagari_QH_main_calibration.m`
- Core algorithms in `manuscripts/`:

- `algorithm_call_default.m`: Hansen’s CMA-ES implementation
- `algorithm_call_sep.m`: Separable CMA-ES implementation
- `stationary_dist.m`: Stationary distribution computation
- `Lossfunction.m`: Loss function implementation

Key features:

- Specialized value function (`valf2`)
- Extended grid policy extrapolation
- Sophisticated grid boundary handling
- Gradient injection for optimization
- Two-state Markov transitions

7.5 Additional Models (Not in Paper)

7.5.1 Brock-Mirman Model

Located in: `optimal growth --- Brock-Mirman --- not in paper/`

Implementation files:

- Main model: `Brock_Mirman_model.m`
- Algorithm: Uses standard CMA-ES implementation

Key features:

- Standard value function (`valf`)
- Basic policy extrapolation
- Stochastic transitions
- Identity matrix covariance initialization

8 Code Organization by Model Type

8.1 Deterministic Optimal Growth/Social Planner Models

- Location: `optimal growth --- deterministic --- Section 3/`
- Main demonstration script: `optimal_growth.m`
- Steady state computations: `steady states --- Online Appendix Section 1/`

8.2 Stochastic Optimal Growth Models (“Quasi-Hyperbolic Brock-Mirman”)

- Location: `optimal growth --- Brock-Mirman --- not in paper/`
- Main demonstration script: `optimal_growth_stochastic.m`

- Note: Simple script for demonstration only (this model does not feature in the paper except abstractly as part of the general model in Section 2)

8.3 Deterministic Consumption-Saving Models

- Location: `determ cons sav --- Section 4/`
- Main demonstration scripts:
 - `determ_cons_sav0_93.m`
 - `determ_cons_sav0_85.m`
 - `determ_cons_savmgr.m`
 - `determ_cons_sav0_85largerdelta.m`

8.4 Bewley-Aiyagari Models

- IID Models (Section 1 and 5)
 - Location: `Bewley-Aiyagari iid --- Sec 1 and Sec 5/`
 - Near-RBC Example (Section 1):
 - * `aiyagari_Sec1_PanelA.m`
 - * `aiyagari_Sec1_PanelB.m`
 - Basic Comparative Statics and Mandatory Saving Schemes (Section 5):
 - * `aiyagari_iid.m`
 - * `aiyagari_iidsm.m`
- Two-State Models (Section 5)
 - Location: `Bewley-Aiyagari general two-state --- Sec 5/`
 - Main calibration script:
 - * `aiyagari_QH_main_calibration.m`

9 Code Placement by Figure

9.1 Section 1: Near-RBC Example

- Figure 1.A: `Bewley-Aiyagari iid --- Sec 1 and Sec 5/`
`Near RBC Textbook Example --- Sec 1/aiyagari_Sec1_PanelA.m`
- Figure 1.B: `Bewley-Aiyagari iid --- Sec 1 and Sec 5/`
`Near RBC Textbook Example --- Sec 1/aiyagari_Sec1_PanelB.m`

9.2 Section 3: Deterministic Optimal Growth

- Figure 4.A: optimal growth --- deterministic --- Section 3/optimal_growth.m
- Figure 4.B: optimal growth --- deterministic --- Section 3/optimal_growth_10it.m

9.3 Section 4: Deterministic Consumption-Saving Models

- Figure 6.A: determ cons sav --- Section 4/determ_cons_sav0_93.m
- Figure 6.B: determ cons sav --- Section 4/determ_cons_sav0_85.m
- Figure 6.C: determ cons sav --- Section 4/determ_cons_sav1_1.m
- Figure 6.D: determ cons sav --- Section 4/determ_cons_savmgr.m
- Figure 7.A: determ cons sav --- Section 4/determ_cons_sav0_93.m
- Figure 7.B: determ cons sav --- Section 4/determ_cons_sav0_85largerdelta.m
- Figure 8: determ cons sav --- Section 4/determ_cons_sav0_85.m

9.4 Section 5: Bewley-Aiyagari Models

- Figures 9.A–9.B:
Bewley-Aiyagari iid --- Sec 1 and Sec 5/
Aiyagari iid and mandatory sav schemes --- Sec 5/aiyagari_iid.m
- Figures 10.A–10.D:
Bewley-Aiyagari general two-state --- Sec 5/aiyagari_QH_main_calibration.m
- Figure 11.A:
Bewley-Aiyagari iid --- Sec 1 and Sec 5/
Aiyagari iid and mandatory sav schemes --- Sec 5/aiyagari_iid.m
- Figure 11.B:
Bewley-Aiyagari iid --- Sec 1 and Sec 5/
Aiyagari iid and mandatory sav schemes --- Sec 5/aiyagari_iidsm.m